Lab 2 **BIOS** and Exceptions

1. C can use functions defined in assembly, simultaneously, assembly can use functions defined in C
2. In `crt0.S`, the `.global` directive exports `_vectors`, and `exception.h` declares it with `extern` so C code can reference it (the compiler knows that `_vectors` is defined elsewhere)
3. As can be seen, each element in the vector is defined as `.word.` `.word` means "relocations." At the moment, we are writing `.word bus_error_handler` in `crt0.s`, and the assembler can't know the final numeric address yet, so the assembler emits a relocation ("fill in the address of `bus_error_handler` later"). The linker resolves that by matching symbol names; so **32-bit slot (our chip is in 32 bit system, so address is 32 bits)** ends up containing the actual address `bus_error_handler()` in `exception.c`
4. An instruction in `main` triggers an exception → the **hardware** jumps directly to the corresponding slot in the exception vector table (_error) → that slot executes `l.j _exception_handler` followed by `l.nop`, transferring control to the shared `_exception_handler` → `_exception_handler` uses the SPR exception code to look up `_vectors` and invokes the actual expection handler. Note: the order of exceptions in `_vectors` need to be fixed, otherwise SPR exception code represents to wrong offset of `_vectors` .

2.4 Exception Vectors

- **In the manual, we were told to overwrite the _vectors[] to add our own exception handlers. HOWEVER, this is INCORRECT.**
  The correct way is to use `__weak()` to specify the functions that could be overwritten.
  Check 2.5.5 - 2.5.6 for the correct overriding of the exceptions handlers.

Question 2.4.1:

1. Read the relevant sections of the OpenRISC 1000 Architecture Manual to learn more about (1) exception classes and (2) exception processing. What is the role of EPC (a special-purpose CPU register)?

Exception	Priority	EPCR (no delay slot)	EPCR (delay slot)	EEAR
Reset	1	-	-	-
Bus Error	4 (insn) 9 (data)	Address of instruction that caused exception	Address of jump instruction before the instruction that caused exception	Load/ store/fetch virtual EA
Data Page Fault	8	Address of instruction that caused exception	Address of jump instruction before the instruction that caused exception	Load/store virtual EA
Instruction Page Fault	3	Address of instruction that caused exception	Address of jump instruction before the instruction that caused exception	Instruction fetch virtual EA
Tick Timer	12	Address of next not executed instruction	Address of just executed jump instruction	-
Alignment	6	Address of instruction that caused exception	Address of jump instruction before the instruction that caused exception	Load/store virtual EA
Illegal Instruction	5	Address of instruction that caused exception	Address of jump instruction before the instruction that caused exception	Instruction fetch virtual EA
External Interrupt	12	Address of next not executed instruction	Address of just executed jump instruction	-
D-TLB Miss	7	Address of instruction that caused exception	Address of jump instruction before the instruction that caused exception	Load/store virtual EA
I-TLB Miss	2	Address of instruction that caused exception	Address of jump instruction before the instruction that caused exception	Instruction fetch virtual EA
Range	10	Address of instruction that caused exception	Address of jump instruction before the instruction that caused exception	-

Exception	Priority	EPCR (no delay slot)	EPCR (delay slot)	EEAR
System Call	7	Address of next not executed instruction	Address of just executed jump instruction	-
Floating Point	11	Address of next not executed instruction	Address of just executed jump instruction	-
Trap	7	Address of instruction that caused exception	Address of jump instruction before the instruction that caused exception	-

Table 6-3. Values of EPCR and EEAR After Exception

Question 2.4.2

2. Open `support/src/crt0.s` in the text editor. Where in this file are the exception vectors *defined*? What is the purpose of the `.global _vectors` directive? You can refer to the GNU Assembler Manual.

7.41 .global *symbol*, .globl *symbol*

`.global` makes the symbol visible to `ld`. If you define *symbol* in your partial program, its value is made available to other partial programs that are linked with it. Otherwise, *symbol* takes its attributes from a symbol of the same name from another file linked into the same program.

Both spellings ('`.globl`' and '`.global`') are accepted, for compatibility with other assemblers.

On the HPPA, `.global` is not always enough to make it accessible to other partial programs. You may need the HPPA-only `.EXPORT` directive as well. See Section 9.15.5 [HPPA Assembler Directives], page 176.

- C can use functions defined in assembly; simultaneously, assembly can use functions defined in C. In `crt0.S`, the `.global` directive exports `_vectors`, and `exception.h` declares it with `extern` so C code can reference it (the compiler knows that `_vectors` is defined elsewhere)

```
.global _vectors
_vectors:
    .word       _start
.ifdef __OR1300__
    .word       bus_error_handler
    .word       data_page_fault_handler
    .word       instruction_page_fault_handler
    .word       tick_timer_handler
    .word       allignment_exception_handler
    .word       illegal_instruction_handler
    .word       external_interrupt_handler
    .word       dtlb_miss_handler
    .word       itlb_miss_handler
    .word       range_exception_handler
    .word       system_call_handler
    .word       break_point_handler
    .word       trap_handler
    .word       my_exception_handler

.else
    .word       i_cache_error_handler
    .word       d_cache_error_handler
    .word       external_interrupt_handler
    .word       illegal_instruction_handler
    .word       system_call_handler
    .word       my_exception_handler
.endif
```

- The exception vectors are defined at the end of the `crt0.s`
- As can be seen, each element in the vector is defined as `.word.` `.word` means "relocations." At the moment, we are writing `.word bus_error_handler` in `crt0.s`, and the assembler can't know the final numeric address yet, so the assembler emits a relocation ("fill in the address of `bus_error_handler` later"). The

linker resolves that by matching symbol names; so **32-bit slot (our chip is in 32 bit system, so address is 32 bits)** ends up containing the actual address `bus_error_handler()` in `exception.c`

In simple terms, the exception handler `_vectors` is defined in `crt0.s`, and this `_vectors` makes reference to exception handler functions defined in `exception.c`. We use `.global` in `crt0.s` to make `_vectors` accessible to other programs; this is why in exception.h we have `extern`, informing the assembler that `_vectors` is defined elsewhere.

```
extern exception_handler_t _vectors[EXCEPTION_COUNT];
```

This is also why we are able to use `_vectors` in `exceptions.c` as below, however this method is INCORRECT (it works but will cause some weird issue, like the ???? we saw). The correct way of doing this is to overwrite `__weak`

```
_vectors[11] = system_call_handler; //overwrite the system call handler with ou
rs
```

Question 2.4.3

3. What does the `_exception_handler` do in `crt0.s`? Explain the `l.rfe` instruction by referring to the OpenRISC 1000 Architecture Manual.

```
_exception_handler:
    l.addi      r1,r1,-124
    l.sw        0x00(r1),r2
    l.sw        0x04(r1),r3
.ifdef __OR1300__
    l.addi      r3,r0,0x200
    l.mtspr     r0,r3,0xF800 # st
.endif
    l.sw        0x08(r1),r4
    l.sw        0x0C(r1),r5
    l.sw        0x10(r1),r6
    l.sw        0x14(r1),r7
    l.sw        0x18(r1),r8
    l.sw        0x1C(r1),r9
    l.sw        0x20(r1),r10
    l.sw        0x24(r1),r11
    l.sw        0x28(r1),r12
    l.sw        0x2C(r1),r13
    l.sw        0x30(r1),r14
    l.sw        0x34(r1),r15
    l.sw        0x38(r1),r16
    l.sw        0x3C(r1),r17
    l.sw        0x40(r1),r18
    l.sw        0x44(r1),r19
    l.sw        0x48(r1),r20
    l.sw        0x4C(r1),r21
    l.sw        0x50(r1),r22
    l.sw        0x54(r1),r23
    l.sw        0x58(r1),r24
    l.sw        0x5C(r1),r25
    l.sw        0x60(r1),r26
    l.sw        0x64(r1),r27
    l.sw        0x68(r1),r28
    l.sw        0x6C(r1),r29
    l.sw        0x70(r1),r30
    l.sw        0x74(r1),r31
```

The `_exception_handler` firstly allocates a space on the stack to store all the values in the general-purpose register by doing **l.addi r1,r1,-124.**

```
l.addi r1, r1, -124   //allocate a space of 4 bytes * 31 on the stack
l.sw   0x00(r1), r2   //stack[r1] = r2
l.sw   0x04(r1), r3   //stack[r1+4] = r2
```

```
/*
Q: What does l.sw instruction do?
l.sw I(rA),rB

EA ← exts(I) + rA[31:0]
(EA)[31:0] ← rB[31:0]

... essentially stores the value in rB inside the address location (I + rA)
*/
```

```
l.mfspr     r31,r0,0x12
l.slli      r31,r31,2
l.movhi     r30,hi(_vectors)
l.ori       r30,r30,lo(_vectors)
l.add       r30,r30,r31
l.lwz       r31,0x0(r30)
l.jalr      r31
l.nop
```

Here is a breakdown of what each line does:

- `l.mfspr r31,r0,0x12`:
  Reads the value of Special Purpose Register (SPR) 0x12, which stores the Exception Index (referring to the *vectors[])* into general-purpose register r31. Note : On your platform, SPR 0x12 is **not** used as the PPC (PC mapped to SPR space), which **differs from the OpenRISC documentation**.
- `l.slli r31,r31,2`:
  Performs a logical shift left of the value in r31 by 2 bits (essentially r31 ∗ 2^2). Since the vector table is an array of 32-bit words (4 bytes), this multiplies the exception vector index by 4 to get the correct byte offset into the table.
- `l.movhi r30,hi(_vectors)`:
  Loads the upper 16 bits of the memory address of the _vectors table into register r30.

- `l.ori r30,r30,lo(_vectors)`:
  Loads the lower 16 bits of the memory address of the _vectors table into register r30 and combines it with the upper 16 bits from the previous instruction. This fully loads the base address of the exception vector table into r30.

- `l.add r30,r30,r31`:
  Adds the calculated offset (in r31) to the base address of the vector table (in r30). **The result is a pointer to the specific vector table entry for the current exception**.

- `l.lwz r31,0x0(r30)`:
  Loads the 32-bit word at the memory address stored in r30 (the vector table entry) into register r31. This word contains the address of the specific exception handler function (e.g., bus_error_handler, external_interrupt_handler, etc.).

- `l.jalr r31`:
  This is a "Jump And Link Register" instruction. It jumps to the address now held in r31 (the specific handler function) and stores the return address in the link register (r9), though it isn't used in this context. The exception handler then executes.

- `l.nop`:
  A nop-operation instruction that serves as a delay slot. On OpenRISC, certain branch instructions require a delay slot, where the instruction following the branch is executed before the branch takes effect.

```
l.lwz       r2,0x00(r1)
l.lwz       r3,0x04(r1)
l.lwz       r4,0x08(r1)
l.lwz       r5,0x0C(r1)
l.lwz       r6,0x10(r1)
l.lwz       r7,0x14(r1)
l.lwz       r8,0x18(r1)
l.lwz       r9,0x1C(r1)
l.lwz       r10,0x20(r1)
l.lwz       r11,0x24(r1)
l.lwz       r12,0x28(r1)
l.lwz       r13,0x2C(r1)
l.lwz       r14,0x30(r1)
l.lwz       r15,0x34(r1)
l.lwz       r16,0x38(r1)
l.lwz       r17,0x3C(r1)
l.lwz       r18,0x40(r1)
l.lwz       r19,0x44(r1)
l.lwz       r20,0x48(r1)
l.lwz       r21,0x4C(r1)
l.lwz       r22,0x50(r1)
l.lwz       r23,0x54(r1)
l.lwz       r24,0x58(r1)
l.lwz       r25,0x5C(r1)
l.lwz       r26,0x60(r1)
l.lwz       r27,0x64(r1)
l.lwz       r28,0x68(r1)
l.lwz       r29,0x6C(r1)
l.lwz       r30,0x70(r1)
l.lwz       r31,0x74(r1)
l.addi      r1,r1,124
```

At the end of the exception handler, it then restore all the values stored on the stack back to the general-purpose registers

```
.endif
    l.rfe
    l.nop
```

l.rfe Return From Exception l.rfe

31 26	25 0
opcode 0x9	reserved
6 bits	26 bits

Format:

```
l.rfe
```

Description:

Execution of this instruction partially restores the state of the processor prior to the exception. This instruction does not have a delay slot.

32-bit Implementation:

```
PC ← EPCR
SR ← ESR
```

64-bit Implementation:

```
PC ← EPCR
SR ← ESR
```

Exceptions:

None

Question 2.4.4

4. Open `support/include/exception.h` in the text editor. This header file allows interfacing with the exception vector from the C code. Notice that each vector is *declared* as:

```c
typedef void (*exception_handler_t)(void); /* function pointer */
extern exception_handler_t _vectors[EXCEPTION_COUNT];
```

What is the purpose of the `extern` keyword? How does `extern` relate to the `.global _vectors` directive?

`extern` - static storage duration and external linkage (unless already declared internal)

- Specifies that the function is defined elsewhere

Check Here for better explanation

Question 2.4.5: try to generate an exception.

volatile

- this keyword prevents the compiler from optimizing code
- hint to the implementation to **avoid aggressive optimization involving the object** because the value of the object might be changed by means undetectable by an implementation

5. Consider the following code snippet:

```
int *addr = (int *) 0x100; // choose whatever address you want
int data;
asm volatile("l.lwz %[d], 0(%[s])" : [d]"=r"(data) : [s]"r"(addr));
```

l.lwz Load Single Word and Extend with Zero l.lwz

31 26	25 . . . 21	20 . . . 16	15 0
opcode 0x21	D	A	I
6 bits	5 bits	5 bits	16bits

Format:

```
l.lwz rD,I(rA)
```

Description:

The offset is sign-extended and added to the contents of general-purpose register rA. The sum represents an effective address. The single word in memory addressed by EA is loaded into the low-order 32 bits of general-purpose register rD. High-order bits of general-purpose register rD are replaced with zero.

32-bit Implementation:

```
EA < - exts(Immediate) + rA[31:0]
rD[31:0] < - (EA)[31:0]
```

64-bit Implementation:

```
EA < - exts(Immediate) + rA[63:0]
rD[31:0] < - (EA)[31:0]
rD[63:32] < - 0
```

Exceptions:

```
TLB miss
Page fault
Bus error
Alignment
```

So **how does the OpenRISC instruction work in particular**?

The instruction is: `"l.lwz %[d], 0(%[s])"`

Part	Role	C Variable Mapping
`l.lwz`	**Instruction**	Load Word Zero-Extended. It loads 4 bytes from memory.
`%[d]`	**Destination Register**	The register that will hold the result. Mapped to the C variable `data`.
`0(%[s])`	**Source Address**	The memory address to load from. It means: Base Address (`%[s]`) + Offset (`0`).
`%[s]`	**Source Register**	The register holding the base memory address. Mapped to the C variable `addr` (which is `0x100`).

- `"r"` — **input** in a register
  The compiler puts the C value into **some GPR** for the asm to *read*.

  c Copy code
  ```c
  int a=1,b=2,sum;
  asm("l.add %0,%1,%2" : "=r"(sum) : "r"(a), "r"(b));
  // "r"(a), "r"(b) are inputs
  ```
- `"=r"` — **output (write-only)** in a register
  The asm writes the result to a GPR; the compiler then copies it to your C variable.

  c Copy code
  ```c
  int out;
  asm("l.mul %0,%1,%2" : "=r"(out) : "r"(x), "r"(y));
  ```

[Type 1]: Below is a code example of generating Illegal Instruction error

▼ **Edwin's result**

→ (int*)0x100 was able to run without exception, and (int*)0x101 was not

... because 0x101 is not 4 bytes aligned

→ the program should return alignment exception, however we got illegal_instruction.

```
__weak void illegal_instruction_handler() {
    puts("???? ");
}
```

from support/src/exceptions

```
while (1) {
//  perf_print_time(PERF_COUNTER_RUNTIME, "runtime");
//  perf_print_cycles(PERF_COUNTER_RUNTIME, "runtime");

 int *addr = (int*)0x100;
 int data;
 asm volatile ("l.lwz %[d], 0(%[s])" : [d]"=r"(data) :[s]"r"(addr));

 for (volatile int i = 0; i < 1000000; ++i) ;
}
perf_stop();

return 0;
}
```

```
*c Compare program loaded in SDRAM with SPI
*r Run program stored in SPI-Flash
*q Toggle stack pointer from SDRAM (default) w
*h This helpscreen

Download: done
Jumping to programm
Hello, world from main (src/exceptions.c:15)!
no swap: 0xdead, swap_u16: 0xadde
no swap: 0xdeadbeef, swap_u32: 0xefbeadde

Clear   Hex output   Logging to:   /hom
Device:  FTDI Dual RS232-HS @ttyUSB1  Connection:
```

The code above should work normally,

- `int *addr = (int*)0x100` initializes an absolute int pointer position at memory address 0x100, which is 256
- in `asm volatile()` we then load the value stored in memory address 256 to data

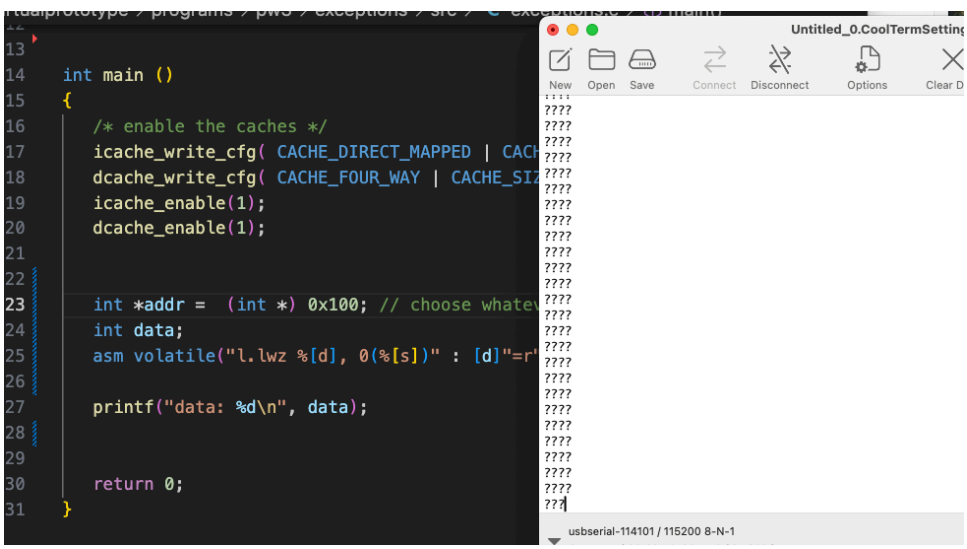
```
asm volatile ("l.lwz %[d], 0(%[s])": [d]"=r"(data) :[s]"r"(addr));

/* "r" we want the compiler to find a random General-Purpose Register an
d load the
   value inside addr to this GPR
*/
/* "=r" we then want the compiler to find a random GPR to store the value
inside
   (addr + 0) to this GPR, and then loads the value inside this GPR to data
*/

// Thus, eventually, we will be able to print the values of data, as shown in
[Type 2]
```

- Notice that 256 is 4 bytes aligned, which should be the case, as we want the memory address to store an integer
- Meaning that if we make 0x100 → 0x104, the program should still work. And if we make it 0x101 (not 4 bytes aligned), then the program should return an error.
- As can be seen below, the program is indeed returning "????", indicating that it cannot understand the instruction

- Mr.Kluter mentioned that normally it is **ILLEGAL** to do something like `(int*) 0x100` in the `main()`. This potentially overwrites some reserved memory space by CPU. And you will get something like below.



- Thus, the solution to this problem is to pre-reserve a memory space using `global`, as shown below

```
int k =6;

int main ()
{
  /* enable the caches /
  icache_write_cfg( CACHE_DIRECT_MAPPED | CACHE_SIZE_2K | CACHE_
REPLACE_FIFO );
  dcache_write_cfg( CACHE_FOUR_WAY | CACHE_SIZE_2K | CACHE_REPL
ACE_LRU | CACHE_WRITE_BACK );
  icache_enable(1);
  dcache_enable(1);

  int addr = &k; // choose whatever address you want
  *addr = 100;
  int data;
  asm volatile("l.lwz %[d], 0(%[s])" : [d]"=r"(data) : [s]"r"(addr));

  printf("data: %d\n", data);

  return 0;
}
```

- In such way, CPU will allocate a space in the main memory, allowing us to use it freely in `main()`

[Type 2]: Below is a code example of generating Bus Error

Apparently, if you don't put the instruction above in the while loop, you will get the result as below.
The idea is to assign 100 to the memory location of addr, so
memory_location[0x00000004] = 100
and `l.lwz` will put the value stored in memory location **0x00000004** to **data**
This is why we can print out the data as data = 100 after the volatile function

```c
perf_init();

int *addr = (int*) 0x00000004;
*addr = 100;
int data;
asm volatile ("l.lwz %[d], 0(%[s])" : [d]"=r"(data) :[s]"r"(addr));
printf("data: %d\n", data);

printf("Hello, world from %s (%s:%2d)!\n", __func__, __FILE__, __LINE__);
```

```
Input:
Download: done
Jumping to programm
data: 100
Hello, world from main (src/exceptions.c:21)!
no swap: 0xdead, swap_u16: 0xadde
no swap: 0xdeadbeef, swap_u32: 0xefbeadde
runtime                [08] : 00:00:00.000
runtime
```

```c
/* initialize the performance counters */
perf_init();

int *addr = (int*) 0x00000101;
*addr = 100;
int data;
asm volatile ("l.lwz %[d], 0(%[s])" : [d]"=r"(data) :[s]"r"(addr));
printf("data: %d\n", data);

printf("Hello, world from %s (%s:%2d)!\n", __func__, __FILE__, __LINE__);
```

```
Input:
Jumping to programm
bus error!
bus error!
bus error!
bus error!
bus error!
```

Question 2.4.6: try to write exception handler

6. In the `src/exceptions.c`, define your own exception handler for one of the exceptions that you generated. The name of the exception handler should start with `my_`. Modify the exception vectors to enable the new exception handler. You can use the enumeration with symbols `EXCEPTION_*` to index the correct exception vector entry. Do the modification only in `exceptions.c`, never modify `crt0.s` or `exception.c`. Do not forget to `#include <exception.h>`.

- We want our own bus handler, so we created a function `my_bus_error_handler()` in `exceptions.c` defined as type `exception_handler_t`
  so that we can add it to the `_vector[]` defined in exception.h
- Note that `_vector[]` is the vector where we store all the exception handlers

```c
#include <stdio.h>
#include <perf.h>
#include <swap.h>
#include <cache.h>
#include <exception.h>

exception_handler_t my_bus_error_handler(){
    puts("hope we are done: bus error");
}


int main () {
    /* enable the caches */
    icache_write_cfg( CACHE_DIRECT_MAPPED | CACHE_SIZE_2K | CACHE_REPLACE_FIFO );
    dcache_write_cfg( CACHE_FOUR_WAY | CACHE_SIZE_2K | CACHE_REPLACE_LRU | CACHE_WRITE_BACK );
    icache_enable(1);
    dcache_enable(1);
    /* initialize the performance counters */
    perf_init();

    _vectors[1] = my_bus_error_handler();

    int *addr = (int*) 0x101;
    *addr = 100;
    int data;
    asm volatile ("l.lwz %[d], 0(%[s])" : [d]"=r"(data) :[s]"r"(addr));
    printf("data: %d\n", data);

    printf("Hello, world from %s (%s:%2d)!\n", __func__, __FILE__, __LINE__);

    printf("no swap: 0x%04x, swap_u16: 0x%04x\n", 0xDEAD, swap_u16(0xDEAD));
    printf("no swap: 0x%08x, swap_u32: 0x%08x\n", 0xDEADBEEF, swap_u32(0xDEADBEEF));

    perf_start();

    while (1) {
     perf_print_time(PERF_COUNTER_RUNTIME, "runtime");
     perf_print_cycles(PERF_COUNTER_RUNTIME, "runtime");


     for (volatile int i = 0; i < 1000000; ++i) ;
    }
    perf_stop();

    return 0;
}
```

```
Input:
Known RS232 commands:
** Start the program loaded in targ
*p Set programming mode (default
*v Set verification mode
*i Show info on program in target
*t Toggle target between SDRam (
*m Perform simple SDRam memch
*s Check SPI-flash chip
*e Erase SPI-Flash chip
*f Store program loaded in SDRAM
*c Compare program loaded in SDF
*r Run program stored in SPI-Flash
*q Toggle stack pointer from SDRAI
*h This helpscreen

Download: done
Jumping to programm
hope we are done: bus error
p<0x04><0x04><0x04><0x04><0x0
x04><0x04><0x04><0x04><0x04><
><0x04><0x04><0x04><0x04><0x0
x04><0x04><0x04><0x04><0x04><
><0x04><0x04><0x04><0x04><0x0
x04><0x04><0x04><0x04><0x04><
Clear   Hex output   Logg
Device: FTDI Dual RS232-HS @ttyUSB1
```

2.5 System Calls

Question 2.5.1:

1. Make a syscall using the SYSCALL(n) macro defined in `exception.h`. n is the syscall number chiefly used for identifying the syscall type. Choose n as `0xAA` for debugging purposes.

```
l.sys K
```

Description:

Execution of the system call instruction results in the system call exception. The system calls exception is a request to the operating system to provide operating system services. The immediate value can be used to specify which system service is requested, alternatively a GPR defined by the ABI can be used to specify system service.

l.sys **System Call** **l.sys**

31 16	15 0
opcode 0x2000	K
16 bits	16 bits

Format:

```
l.sys K
```

Description:

Execution of the system call instruction results in the system call exception. The system calls exception is a request to the operating system to provide operating system services. The immediate value can be used to specify which system service is requested, alternatively a GPR defined by the ABI can be used to specify system service.

Because an l.sys causes an intentional exception, rather than an interruption of normal processing, the matching l.rfe returns to the next instruction. As this is considered to be the jump itself for exceptions occurring in a delay slot, l.sys should not be placed in a delay slot.

32-bit Implementation:

```
system-call-exception(K)
```

64-bit Implementation:

```
system-call-exception(K)
```

Exceptions:

```
System Call
```

Questions 2.5.2 - 2.5.4:

2. Write your custom handler for the system call exception (whose name should start with `my_`), then modify the exception vector to enable it. The handler should print the value of EPC. `support/include/spr.h` defines helper macros to read to/write from the special purpose registers (SPRs). You can include it by `#include <spr.h>`. Use `printf("0x%08x", ...)` for printing the instructions.

- since execution of `l.sys` results in the system call exception,
  the `system_call_handler()` in `_vectors[11]` will be called.
  Thus the idea is to define our own `my_system_call_handler()` function and overwrite it to
  `_vectors[11]`

  With this setup, when we call `l.sys`, our `my_system_call_handler()` will be called
- Now the question is,
  "How do we extract the syscall number K?"
  <The Idea>
  - For l.sys, EPCR stores the not-executed instruction.
  - We thus use this property to get the argument K of the l.sys instruction
  - We take the PC in EPCR out and assign it to `epc`. This `epc` points to the instruction right after the l.sys
    (Why? We assume this is because, as stated in the manual, the matching l.rfe returns to the next instruction.)
  - To get the PC that actually points to the l.sys, we need to do `epc` - 4.
    This is because each instruction is 4 bytes (32 bits).
  - Once we get the previous PC, which points to l.sys
    we then extract the actual instruction by using the dereferencing in C

```
#define TASK2_5_2     0x0000u
#define TASK2_5_3     0x0001u
#define TASK2_5_4     0x00E0u

void system_call_handler(void){

  uint32_t epc = SPR_READ(SPR_EPC);                // exception program c
ounter

  volatile uint32_t *p = (uint32_t *)epc;          // exception program count
er address
  uint32_t inst_epc = *p;                        // instruction inside the exceptio
```

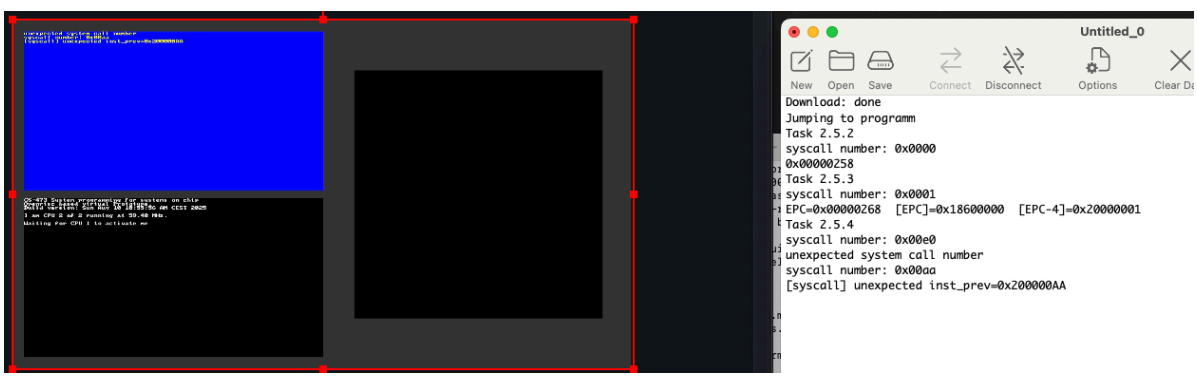
```
n program counter
  volatile uint32_t *epc_prev = (uint32_t *)(epc - 4); // previous program c
ounter address
  uint32_t inst_prev = *epc_prev;                    // instruction inside the prev
ious program counter
                                                     // (this is where the instruction that caus
ed the exception is stored)
  uint16_t sys_call_num = inst_prev & 0xFFFF;        // l.sys K, 16 bits + 16 b
its, we want to get the K
  printf("syscall num: 0x%04x\n", sys_call_num);



  if (sys_call_num == TASK2_5_2) {
    printf("EPC: 0x%08x \n", p);
  }
  else if (sys_call_num == TASK2_5_3){
    printf("[EPC: 0x%08x]=0x%08x\n[EPC-4: 0x%08x]=0x%08x\n", p, in
st_epc,  epc_prev, inst_prev);
  }
  else if (sys_call_num == TASK2_5_4){
    vga_clear();
  }
  else {
    printf("sys call error\n");
  }
}
```

```c
void my_system_call_handler(void)
{
    uint32_t epc = SPR_READ(SPR_EPC);
    volatile uint32_t *p = (uint32_t *)epc;
    uint32_t inst_prev    = *(p - 1);
    uint16_t sysnum = inst_prev & 0xFFFF;
    printf("syscall number: 0x%04x \n",sysnum);

    if(sysnum == TASK2_5_2)
    {
        printf("0x%08x \n",SPR_READ(SPR_EPC));
    }
    else if(sysnum == TASK2_5_3)
    {
        uint32_t inst_epc     = *p;
        printf("EPC=0x%08x  [EPC]=0x%08x  [EPC-4]=0x%08x\n",
                epc, inst_epc, inst_prev);
    }
    else if(sysnum == TASK2_5_4)
    {
        vga_clear();
    }
    else
    {
        printf("[syscall] unexpected inst_prev=0x%08X \n", inst_prev);
    }
}
```



- You can think of it in this way:

  when exception happens...
  memory[0x00000268] = l.sys K
  memory[0x0000026c] = ???
  EPC == 0x0000026c

  Thus, we do EPC - 4, which is 0x00000268 to get l.sys K
  to get the K, which is the least significant 16 bits, we create a 16-bit mask

Notice that the value EPC is storing is an integer, and we want an address so that we could dereference the address to get the content of memory[0x00000268]. This is the reason why, we do `(uint32_t *) epc` to cast the epc into a memory address

- Of course you can have an alternative solution, which is to call the openRisc instruction set directly as below

```
uint32_t instr1,instr2;
asm volatile("l.lwz %[d], 0(%[s])" : [d]"=r"(instr1) : [s]"r"(epc));

epc -= 4;
asm volatile("l.lwz %[d], 0(%[s])" : [d]"=r"(instr2) : [s]"r"(epc));
```

Question 2.5.5:

`__weak` modifier

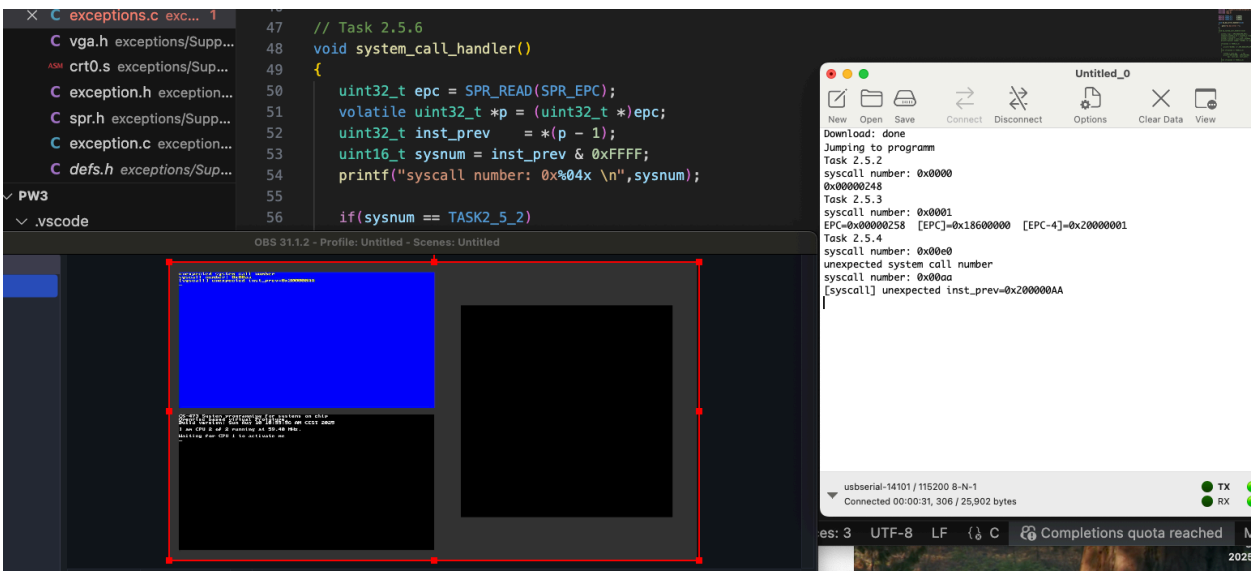
```
/**
 * @brief Defines a weak symbol.
 *
 */
#define __weak __attribute__((weak))
```

- A weak symbol denotes a specially annotated symbol during linking of Executable and Linkable Format (ELF) object files. By default, without any annotation, a symbol in an object file is strong. **During linking, a strong symbol can override a weak symbol of the same name**. This behavior allows an executable to override standard library functions, such as malloc(3). When linking a binary executable, a weakly declared symbol does not need a definition. In comparison, (by default) a declared strong symbol without a definition triggers an undefined symbol link error.
- It's just a compiler-dependent extension to tell the C compiler that an object is weakly declared, as by default objects are strongly declared.
- It is used to provide a default implementation of a function if the user does not write a function(normal, non-weak) with the same name to override the default weak implementation.

- Therefore, you can override exception handlers in exception.c with your own without touching the vector table.

Note: Weak symbols are not mentioned by the C or C++ language standards; as such, inserting them into code is not very portable.

Question 2.5.6:

6. Now, rename your exception handler to `system_call_handler` (the same name as in the `exception.c`). Do not modify the exception vector. Should your code still work given that there are now two symbols with the same name? Why or why not? Test it.

Yes, the code still works after overriding the system_call_handler().